

CHAPTER · DECISION TREE

의사결정나무 (CART) 재귀적 이진 분할 · 가지치기 · 교차검증

공간을 네모로 나누어 예측하는 가장 직관적인 모형 . 어떻게 나누고 , 언제 멈추고 , 어떤 크기의 트리를 고를까 ?

학습 목표

1 트리 모형의 구조

재귀적 이진 분할이 입력 공간을 사각형 영역으로 나누는 방식과 예측값 계산법을 이해한다 .

2 트리 적합 알고리즘

RSS· 지니 불순도를 최소화 하는 분할을 탐욕적 (greedy) 으로 찾는 과정을 설명할 수 있다 .

3 가지치기와 교차검증

Cost-complexity pruning 과 CV 로 최적 크기의 트리를 고를 수 있다 .

4 R 로 실습

rpart() 로 트리를 적합 · 가지치기하고 , 장단점을 바탕으로 결과를 해석할 수 있다 .

트리 모형이란 ?

재귀적 이진 분할 (Recursive Binary Splits)

자식 노드의 불순도 (impurity) 가 가장 작아지도록 데이터를 두 개로 나누는 일을 반복

1 뿌리 노드 (root)

전체 데이터에서 시작하는 맨 위 노드

2 분할 규칙

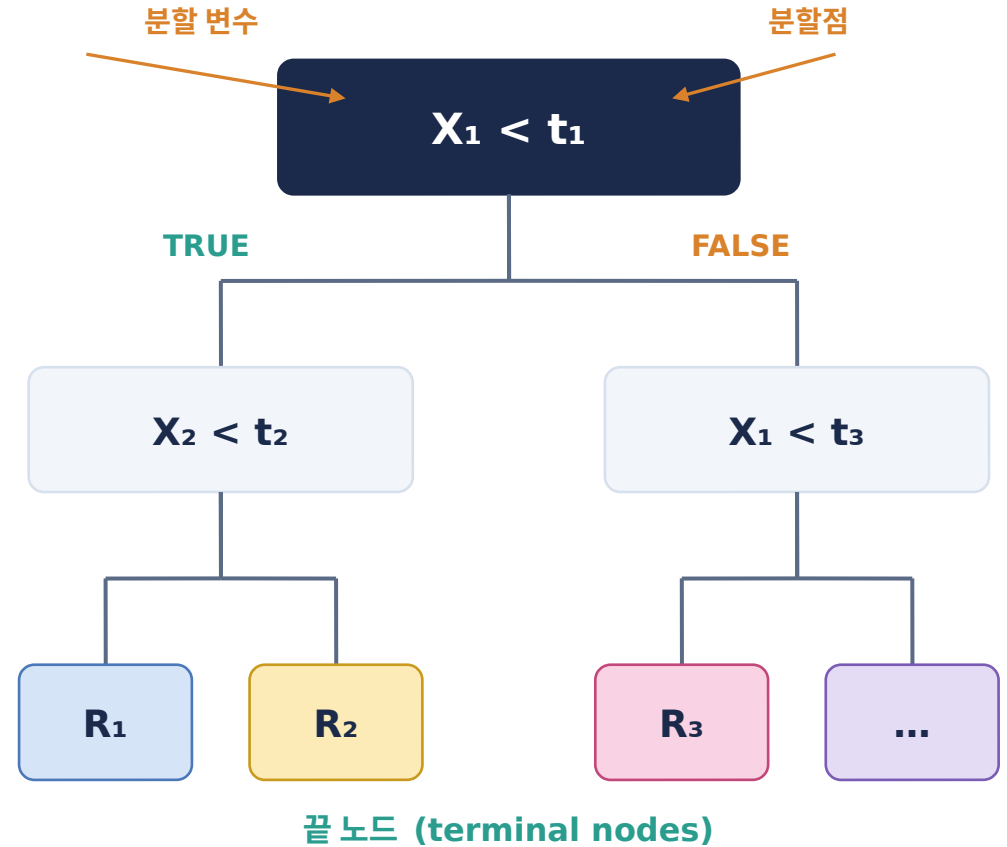
분할 변수 (splitting variable) + 분할점 (splitting point): $X_j < t$

3 방향 규칙

조건이 TRUE 면 왼쪽, FALSE 면 오른쪽 가지로

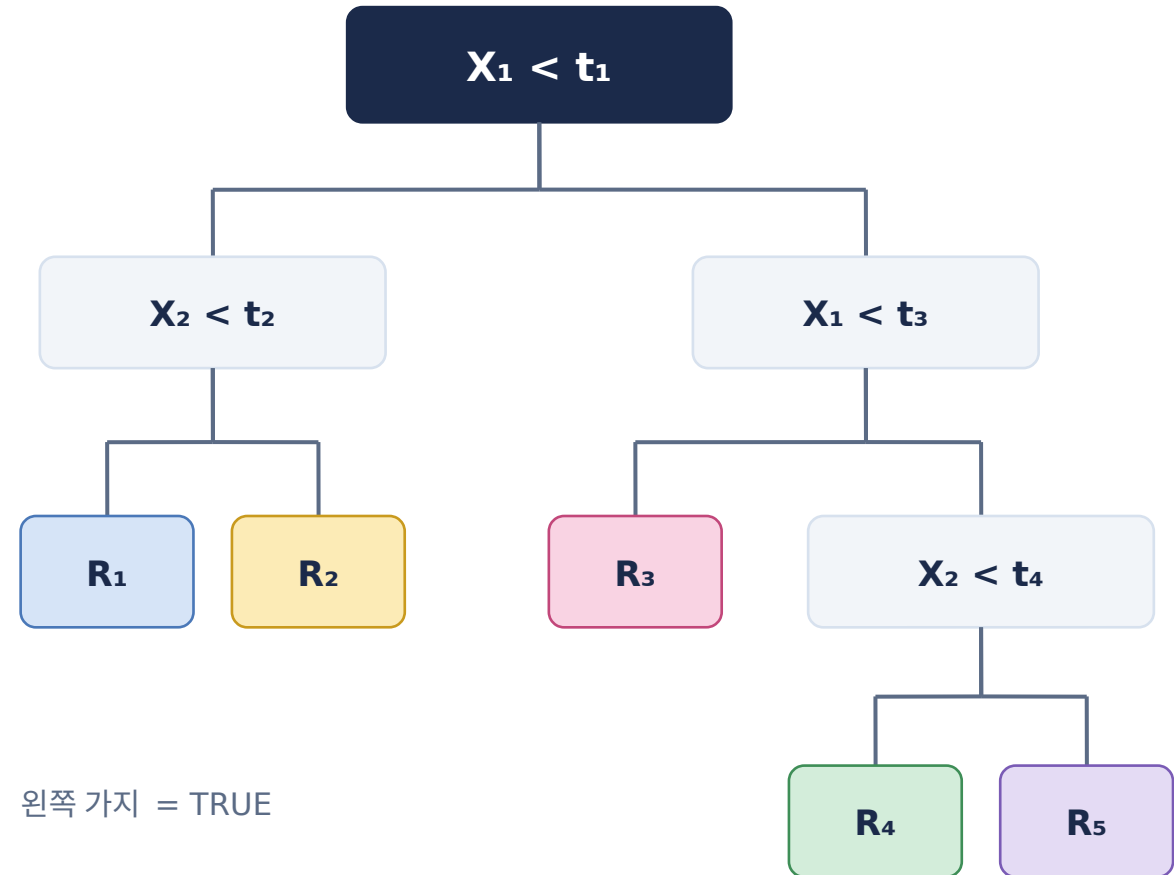
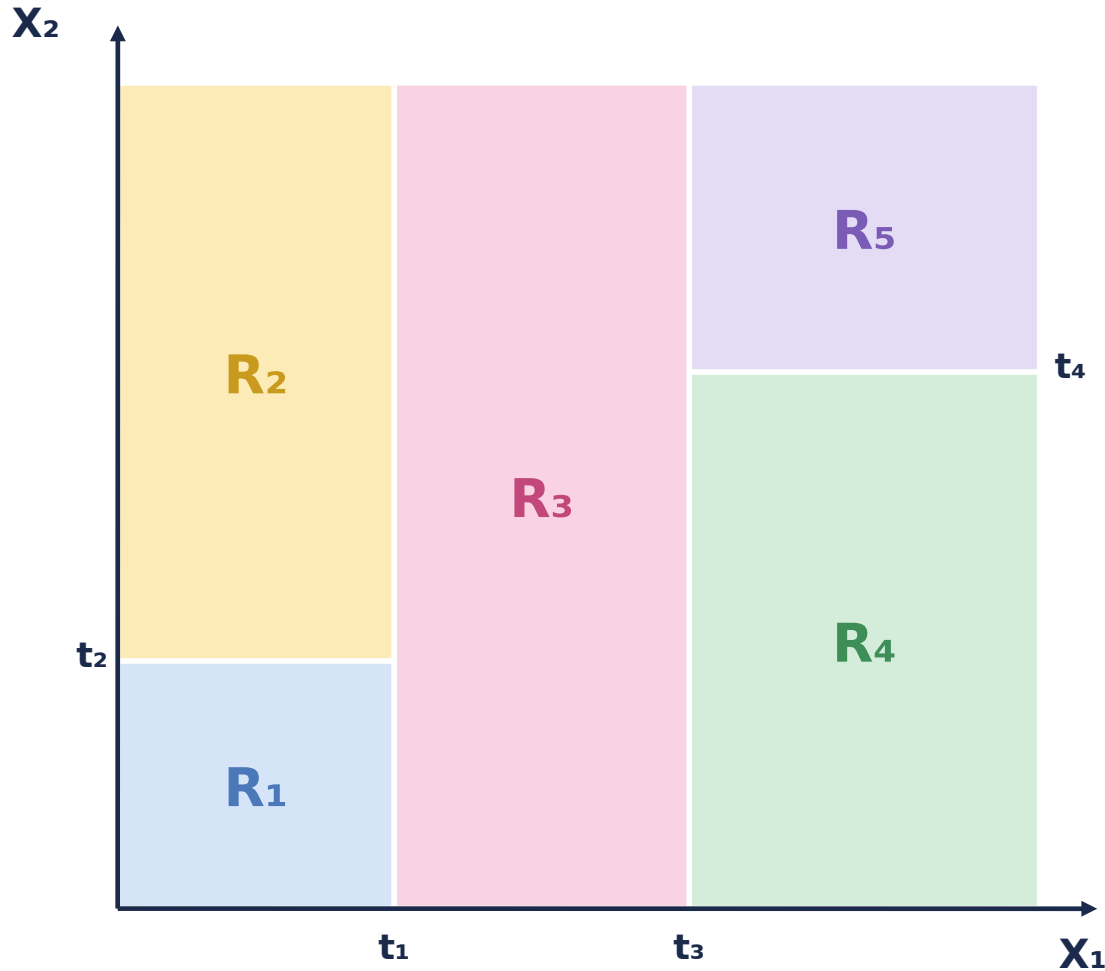
4 끝 노드 (terminal node, leaf)

더 이상 나누지 않는 노드 = 최종 영역 R_m , 여기서 예측



노드 = 데이터의 부분집합, 깊이 (depth) = 뿌리에서 내려온 단계 수

트리 = 입력 공간을 사각형으로 나누기



끝 노드 5 개 = 서로 겹치지 않는 사각형 영역 5 개 ($R_1 \sim R_5$). 모든 경계는 축에 평행하다.

예측값은 어떻게 계산할까 ?

새 관측치를 뿌리에서부터 규칙을 따라 내려보내 , 도착한 끝 노드 R_m 에서 예측

회귀나무 (Regression)

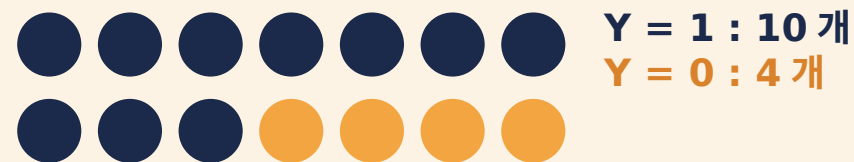
끝 노드에 속한 관측치들의 Y 의 평균

$$\hat{y}(x) = (1/n_m) \sum_{i \in R_m} y_i$$

→ 트리의 예측은 영역마다 일정한 계단 함수 (step function)

분류나무 (Classification)

끝 노드 안에서 다수결 (majority voting)



예측 클래스 = 1

클래스 확률 $P(Y=1) = 10/14 \approx 0.71$ 도 함께 얻음

트리 적합 : 무엇을 최소화할까 ?

목표 : 예측 오차 (MSE·RSS) 가 최소인 , 적당한 크기의 트리를 찾는 것

$$\text{RSS} = \sum_m \sum_{i \in R_m} (y_i - \bar{y}_{R_m})^2 \rightarrow \min$$

문제

가능한 모든 분할 조합 (트리) 을 비교하는 것은 불가능 — 후보 모형이 너무 많다

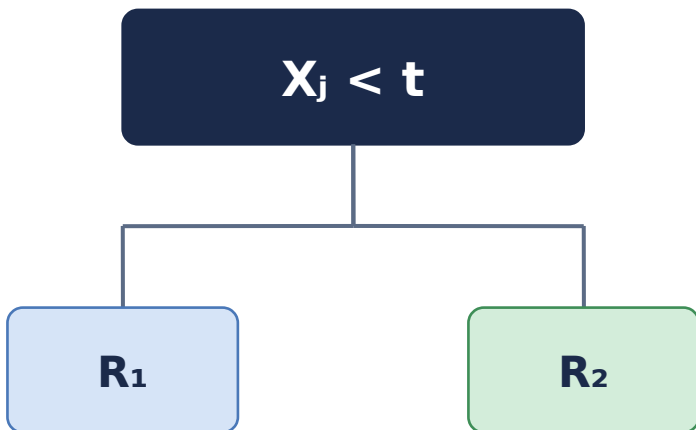
해결 : 탐욕적 탐색 (Greedy search)

매 단계에서 지금 당장 가장 좋은 분할 하나만 선택 (top-down, 되돌아가지 않음)

지난 시간과 연결

stepwise 변수선택과 같은 발상 : 전역 최적 대신 한 걸음씩 최선을 고르는 국소 탐색

한 번의 분할은 어떻게 평가할까 ?



$$R_1 = \{X \mid X_j < t\}, \quad R_2 = \{X \mid X_j \geq t\}$$

회귀나무의 분할 기준 : RSS

$$RSS(j, t) = \sum_{i \in R_1} (y_i - \bar{y}_1)^2 + \sum_{i \in R_2} (y_i - \bar{y}_2)^2$$

$\bar{y}_1, \bar{y}_2$: 각 자식 노드의 평균 . 분할이 정해지면 RSS 는 바로 계산 가능

최선의 분할 = $RSS(j, t)$ 를 최소로 하는 (변수 j , 분할점 t) 의 조합

“자식 노드의 불순도를 최소화” — 회귀에서는 불순도 = 노드 내 제곱오차 (RSS), 분류에서는 지니 지수 · 엔트로피 (뒤에서)

최적 분할 찾기 알고리즘 (변수 $X_1, \dots, X_p$)

1 X_1 정렬

$X_{(1)} \leq X_{(2)} \leq \dots \leq X_{(n)}$ 로 정렬 $\rightarrow$ 이웃한 값 사이가 분할점 후보 ($n-1$ 개)

2 X_1 의 최적 분할

$X_1 < X_{(2)}, X_1 < X_{(3)}, \dots$ 각각의 $RSS_1 \dots RSS_{n-1}$ 계산 $\rightarrow$ 최소인 t_1 저장

3 모든 변수 반복

$X_2, \dots, X_p$ 에 대해 같은 작업 $\rightarrow$ 변수별 최소 RSS

4 Min of Min

p 개의 최소 RSS 중 다시 최소 $\rightarrow$ 첫 번째 분할 결정 (예 : $X_5 < 10$)

5 재귀 반복

나뉜 R_1, R_2 각각에서 ① ~ ④ 를 반복

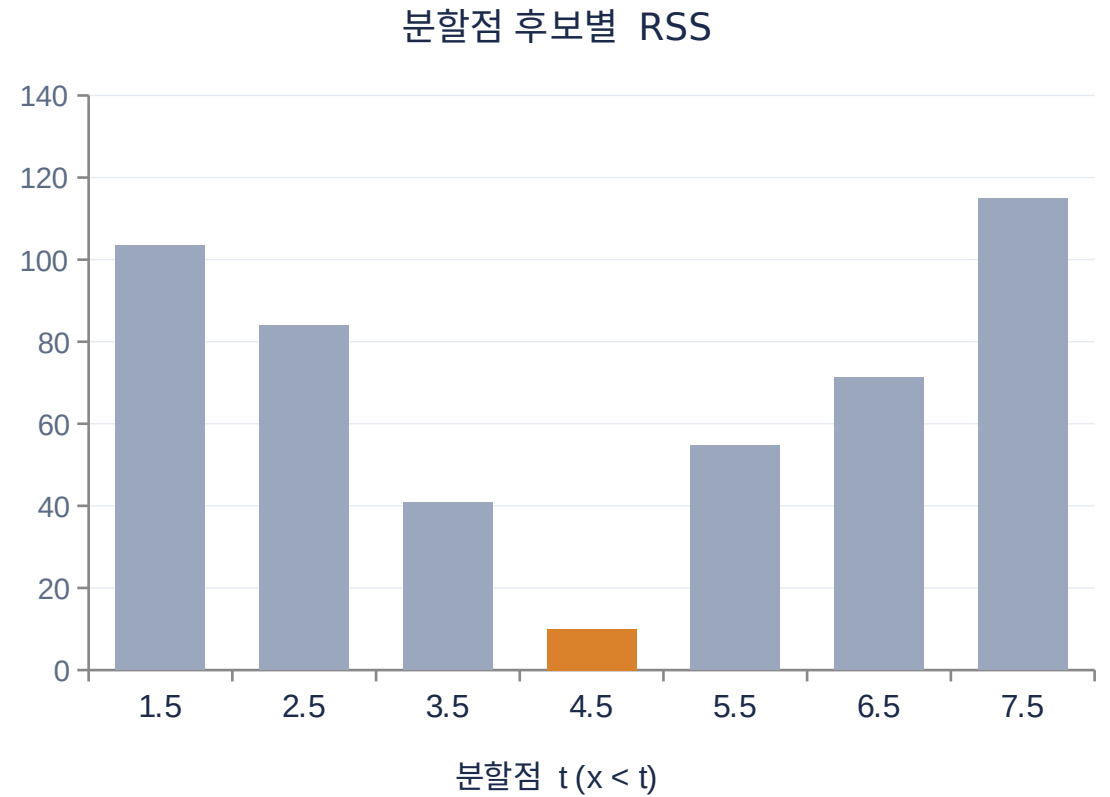
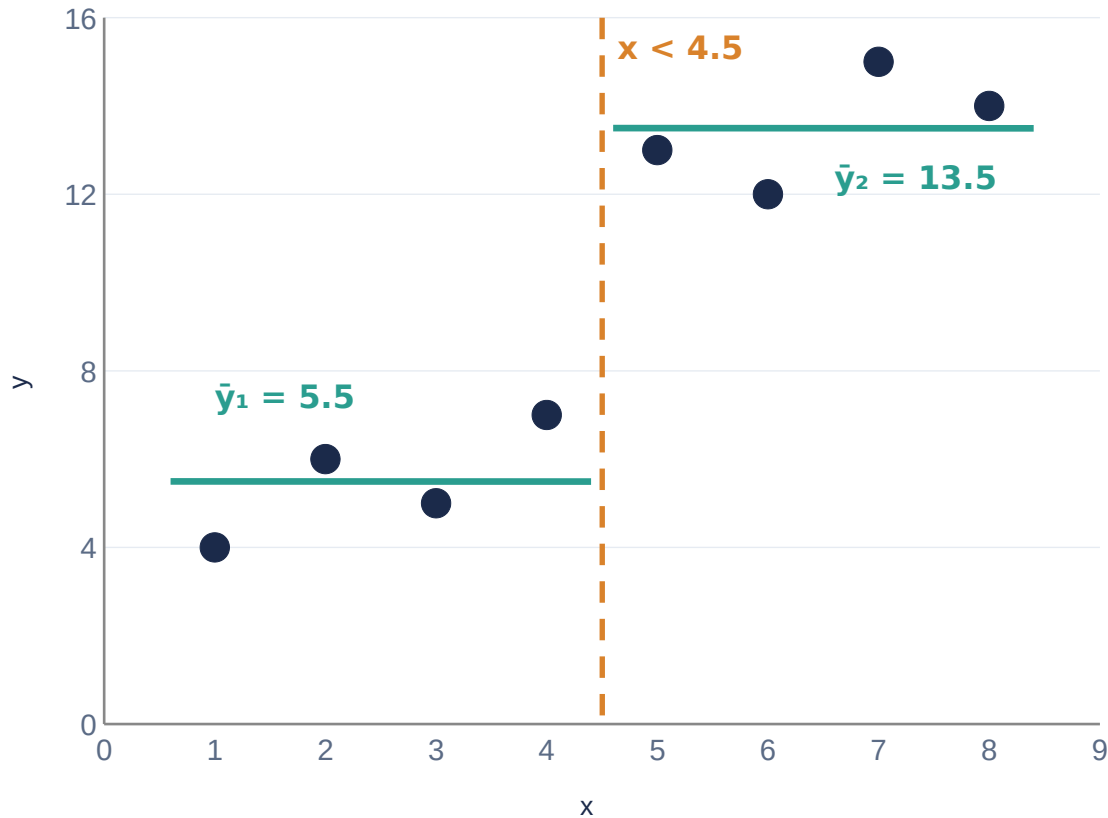
변수	최적 분할점	최소 RSS
X_1	$X_1 < 3.2$	512.4
X_2	$X_2 < 0.7$	498.1
$\vdots$	$\vdots$	$\vdots$
X_5	$X_5 < 10$	401.6
$\vdots$	$\vdots$	$\vdots$
X_p	$X_p < 55$	530.9

가장 작은 값 $\rightarrow$ 첫 분할 : **$X_5 < 10$**

* 표의 수치는 설명용 예시

분할점은 보통 이웃한 두 값의 중간점을 사용

작은 예제 : 8 개 관측치에서 첫 분할 찾기

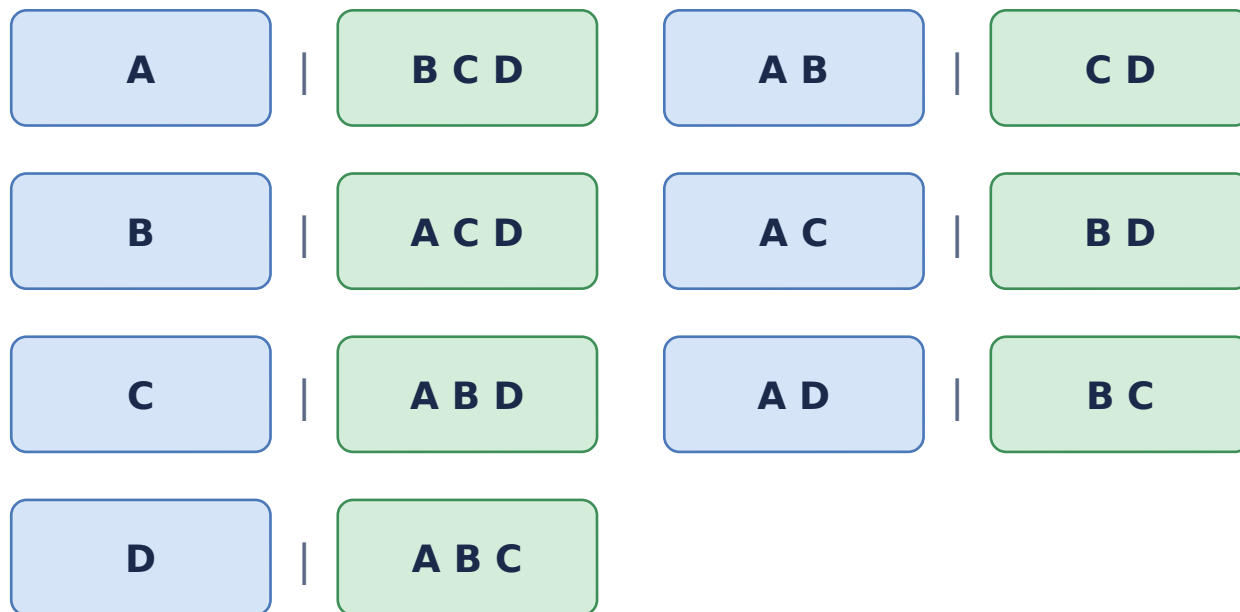


분할 전 $RSS = 138 \rightarrow x < 4.5$ 로 나누면 $RSS = 10$ (최소). 나머지 후보는 모두 더 큰 RSS

범주형 변수는 어떻게 나눌까 ?

수준 (level) 을 두 그룹으로 나누는 모든 방법이 분할 후보

예 : 수준이 A, B, C, D ($k = 4$)



수준이 k 개일 때 분할 후보 수

$$2^{k-1} - 1$$

$k = 4 \rightarrow 2^3 - 1 = 7$ 가지

왜 $2^{k-1} - 1$?

k 개 수준을 두 그룹에 넣는 방법 2^k 에서, 좌 · 우를 바꾼 것은 같은 분할 ($\div 2$), 한쪽이 비는 경우 1 개 제외

참고

k 가 크면 후보가 폭발적으로 증가 $\rightarrow$ 회귀 · 이진 분류에서는 수준을 $\bar{y}$ 순으로 정렬해 $k-1$ 개만 보면 됨

분류나무의 불순도 (impurity)

분류에서는 RSS 대신 노드의 “섞인 정도” 를 최소화 . $\hat{p}_{mk}$ = 노드 m 에서 클래스 k 의 비율

지니 지수 (Gini)

rpart 기본값

$$G = \sum_k \hat{p}_{mk} (1 - \hat{p}_{mk})$$

예시 0.408

엔트로피 (Entropy)

split="information"

$$D = - \sum_k \hat{p}_{mk} \log_2 \hat{p}_{mk}$$

예시 0.863

오분류율

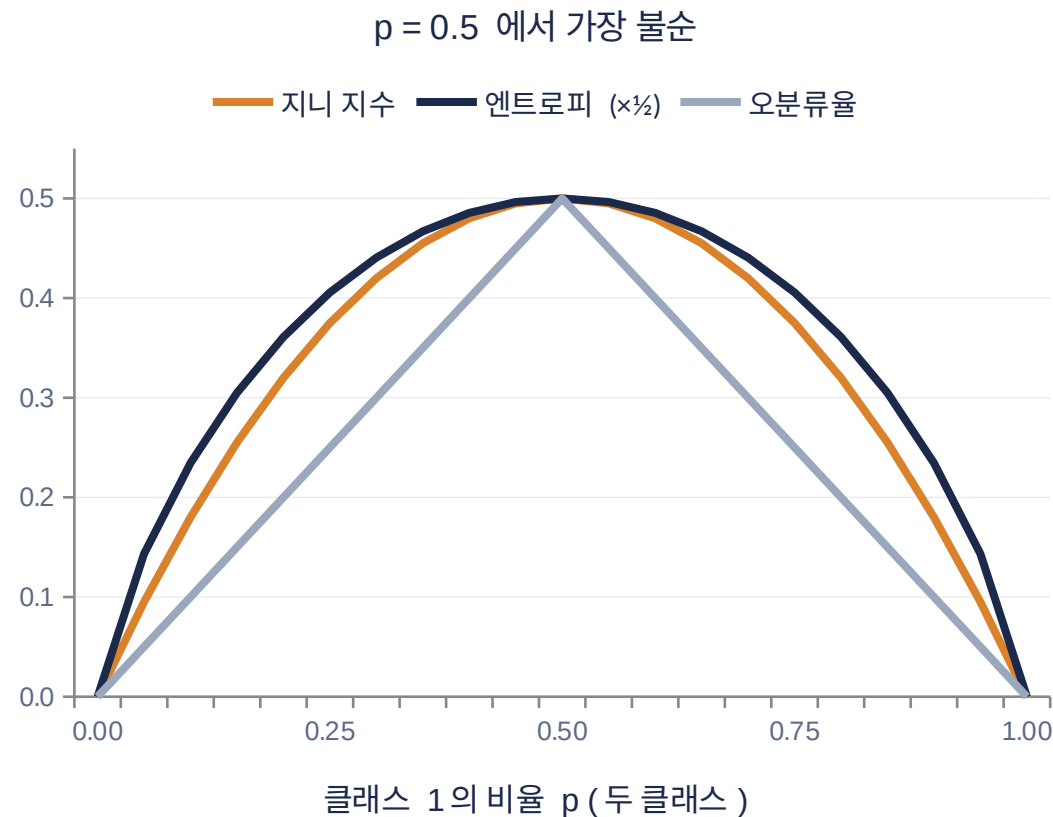
가지치기·평가에 주로 사용

$$E = 1 - \max_k \hat{p}_{mk}$$

예시 0.286

예시 : 앞의 노드 (Y=1: 10 개 , Y=0: 4 개), $\hat{p} = 10/14 \approx 0.714$

순수 노드 (한 클래스만) 면 세 지표 모두 0



언제 분할을 멈출까 ?

1 사전 정지 규칙 (early stopping)

- 끝 노드의 관측치 수가 일정 수 이하면 멈춤 (minsplit, minbucket)
- 최대 깊이 도달 (maxdepth)
- RSS 감소량이 기준보다 작으면 멈춤 (cp)

한계 : 지금은 별 효과 없는 분할 뒤에 좋은 분할이 올 수 있는데 , 그 기회를 놓침 (근시안적)

2 크게 키운 뒤 가지치기 (pruning)

- 먼저 매우 큰 트리 T_0 를 키운다 (느슨한 정지 규칙)
- 필요 없는 가지를 아래에서부터 잘라 부분 트리들을 만든다
- 그중 CV 오차가 가장 작은 트리를 선택

CART 의 표준 방법 : Cost-complexity pruning

트리가 클수록 training RSS 는 계속 줄지만 과대적합 → “적당한 크기” 의 선택이 곧 모형 선택 문제

Cost-complexity pruning (가지치기)

$$C_{\lambda}(T) = \text{RSS}(T) + \lambda \cdot |T|$$

RSS(T)

트리 T의 적합도 (training error)

|T|

끝 노드 (terminal node) 수 = 트리의 복잡도

$\lambda \geq 0$

복잡도 벌점의 크기 (tuning parameter).
이 값이 최소인 트리를 찾는다

T₀

|T| = 12

⊃

T₁

|T| = 8

⊃

T₂

|T| = 5

⊃

T₃

|T| = 3

⊃

뿌리만

|T| = 1

$\lambda = 0$ (가장 큰 트리)

λ 매우 큼

λ 증가 → 가지가 잘려 나가며 중첩된 (nested) 부분 트리 열이 생김 → 최적 λ 는 교차검증으로 선택

교차검증 (Cross-Validation)

모든 예측 모형에서 최적의 모형 (튜닝 모수) 을 찾는 데 쓸 수 있는 방법

5-fold CV: 데이터를 5 개로 무작위 분할 (random partition)

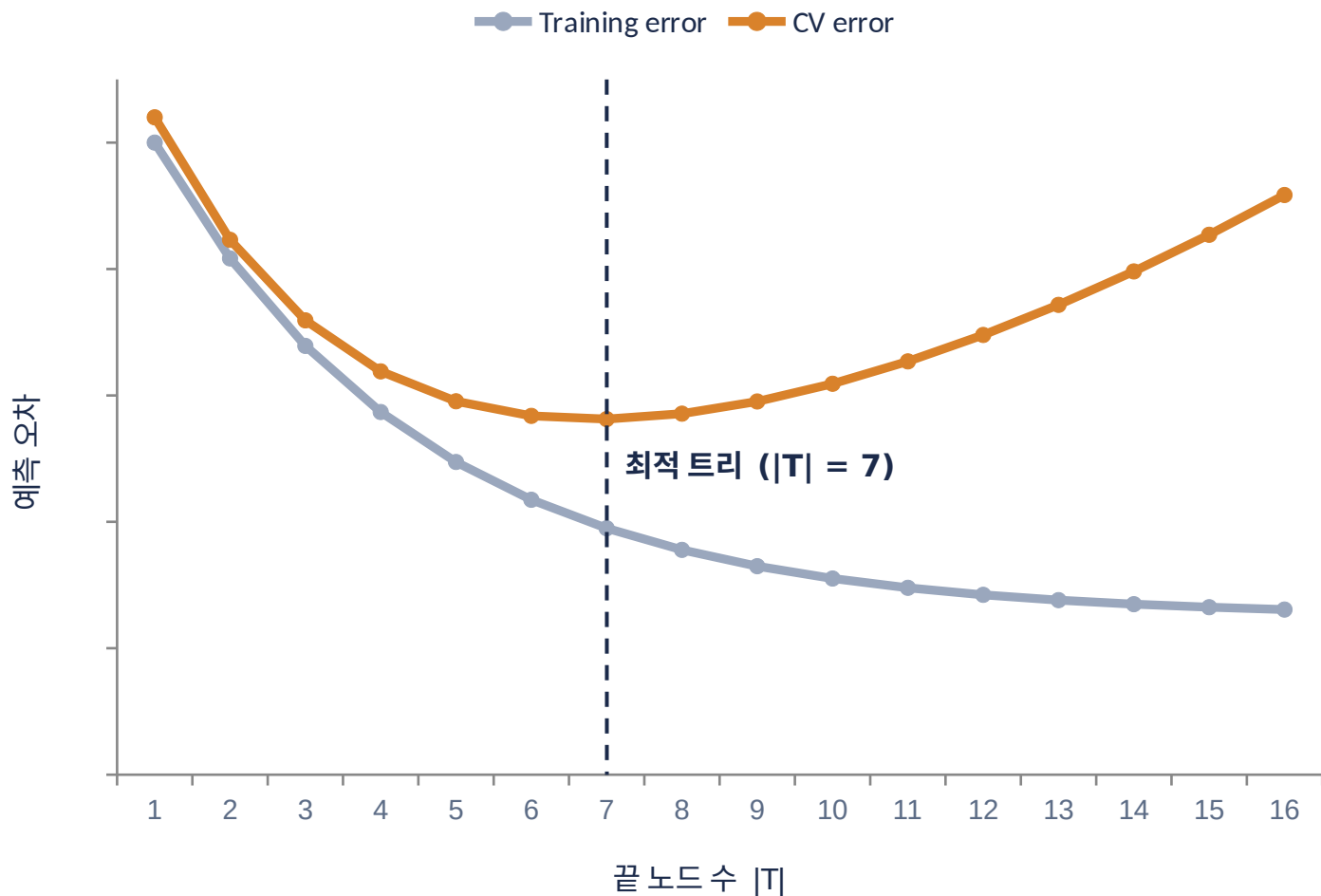
반복 1	Test	Train	Train	Train	Train	→ MSE ₁
반복 2	Train	Test	Train	Train	Train	→ MSE ₂
반복 3	Train	Train	Test	Train	Train	→ MSE ₃
반복 4	Train	Train	Train	Test	Train	→ MSE ₄
반복 5	Train	Train	Train	Train	Test	→ MSE ₅

절차

- P1 을 test 로 , P2~P5 로 모형 적합
→ P1 에서 예측 오차 계산 (회귀 : MSE)
- P2 를 test 로 , P1·P3·P4·P5 로 적합
...
- **5 번 반복한 예측 오차의 평균 = CV error**

CV error = $(1/5) \sum_k \text{MSE}_k$ — 모든 관측치가 한 번씩 test 로 쓰임

CV 로 최적 트리 크기 고르기



CV error 곡선

$|T|$ (또는 λ) 를 바꿔 가며 CV error 계산 $\rightarrow$ U자 모양

최솟값에서 선택

CV error 가 최소인 크기의 트리 = optimal tree, 이 모형을 사용

1-SE 규칙 (실무)

최소 CV error + 1 표준오차 안에 드는 가장 작은 트리 $\rightarrow$ 더 단순하고 안정적

여러 모형 중 최종 모형 고르기 (M1, M2, ..., M5)



Training data 에서

각 모형 (M1: 트리, M2: 선형회귀, ...) 마다 CV error 가 최소가 되도록 튜닝 모수를 정한다



Test data 에서

튜닝된 모형들의 예측 오차를 비교해 가장 작은 모형을 고른다



Final model

이렇게 찾은 모형으로 예측하고, 결과를 해석 · 설명한다

역할 분담

Training data (+CV) → 모형 적합과 튜닝 Test data → 모형 평가 · 비교 (overfitting 모형 회피, 실제 예측 오차 측정)

R 실습 : rpart 로 트리 적합과 가지치기

R CONSOLE

```
> library(rpart)
> set.seed(1234)
# 1) 크게 키우기 (cp를 작게)
> rt <- rpart(a1 ~ ., data=train,
+   control=rpart.control(cp=0.001, minsplit=20, xval=10))
# 2) CV 결과 확인
> printcp(rt); plotcp(rt)
# 3) xerror 최소인 cp로 가지치기
> cp.best <- rt$cptable[which.min(rt$cptable[, "xerror"]), "CP"]
> rt.p <- prune(rt, cp=cp.best)
# 4) test set 예측
> pred <- predict(rt.p, newdata=test)
```

cp

λ 를 뿌리 노드 RSS로 나눈 상대값. 분할이 RSS를 cp 비율 이상 줄여야 채택

xerror / xstd

printcp()의 CV 오차 (상대값)와 그 표준오차 → 1-SE 규칙에 사용

minsplit · minbucket

분할을 시도할 최소 관측치 수 / 끝 노드 최소 관측치 수

분류나무

method="class", predict(..., type="class")

결측치 처리와 변수 중요도

1 대리 분할 (Surrogate split)

- 분할 변수 값이 결측이면, 주 분할과 가장 비슷하게 데이터를 나누는 다른 변수의 규칙을 대신 사용
- 결측치를 채우지 않아도 적합 · 예측 가능
- Part 2 에서 회귀나무에 원본 `algae`(결측 포함) 를 쓸 수 있었던 이유

2 변수 중요도 (Variable importance)

- 각 변수가 분할 (대리 분할 포함) 에서 줄인 불순도 (RSS· 지니) 의 합
- 트리 위쪽에서 쓰인 변수일수록 대체로 중요
- R: `rt$variable.importance`

트리는 변수 척도 변환 · 더미 변수 생성이 필요 없고, 범주형 · 연속형 변수를 섞어 써도 된다

단일 트리의 장점과 한계

장점

- 해석이 쉽다 : 규칙 (if-then) 으로 설명 가능 ,
그림으로 전달
- 변수 간 상호작용과 비선형 관계를 자동으로 포착
- 척도 변환 불필요 , 범주형 변수 · 결측치 처리 용이
- 이상치 · 분포 가정에 비교적 둔감

한계

- 불안정 (high variance): 데이터가 조금만 바뀌어도
트리 구조가 크게 달라짐
- 예측 정확도가 다른 방법보다 낮은 경우가 많음
- 계단 함수 · 축에 평행한 경계 → 매끄러운 / 대각선 관계
표현이 어려움
- 탐욕적 탐색 → 전역 최적 트리를 보장하지 않음

다음 단계 : 여러 트리를 결합해 분산을 줄이는 앙상블 — Bagging, Random Forest, Boosting

SUMMARY

오늘의 핵심 정리

1 트리 = 공간 분할

재귀적 이진 분할로 축에 평행한 사각형 영역을 만들고, 영역 안에서 평균 (회귀) 또는 다수결 (분류) 로 예측

2 탐욕적 적합

모든 변수 · 분할점의 RSS(분류 : 지니) 를 계산해 min of min 을 고르고, 자식 노드에서 반복

3 가지치기 + CV

크게 키운 뒤 $RSS + \lambda|T|$ 로 가지치기, CV error 가 최소인 크기의 트리를 선택

4 최종 모형

Training(+CV) 으로 튜닝, Test 로 모형 비교 → final model 로 예측 · 해석